

Class Activity:

“The SmartCare System Disaster”

Scenario

A mid-sized hospital chain decides to build a **SmartCare Hospital Management System**.

The system should:

- Manage patient records
- Handle appointments
- Process billing
- Provide a doctor dashboard
- Allow mobile access for patients

The CEO wants it completed in **6 months**.

Development Reality:

- Requirements were gathered in only 2 meetings.
- No written SRS document was finalized.
- Developers started coding immediately.
- Client keeps changing features.
- Testing was left for the final 2 weeks.
- No risk analysis was done.
- Budget was cut midway.

After 7 months:

- System crashes during peak hours.
- Doctors refuse to use it.
- Patient data was accidentally exposed.
- Project cost doubled.
- Management calls it a “technical failure.”

Now the fun begins.

Activity Structure

◆ Phase 1: Failure Analysis

1. Identify **at least 5 reasons** why the project failed.
2. Map each failure to:
 - A violated **software quality attribute**
 - A missing or poorly executed **SDLC phase**

Example format:

Problem	Quality Attribute Affected	SDLC Phase Ignored
Data breach	Security	Design / Testing
Doctors refuse system	Usability	Requirements

justify answers logically. No guessing allowed.

◆ Phase 2: Choose the Right SDLC Model

1. Select the most suitable SDLC model:
 - Waterfall
 - Incremental
 - Spiral
 - Agile
 - RAD
2. Justify WHY that model would work better.

consider:

- Changing requirements
- Risk level (healthcare = high risk)
- Time pressure
- Stakeholder involvement

Spoiler alert: There is no perfect answer. But there are weak answers.

◆ Phase 3: Rescue Plan

Being a consultant propose:

- A revised development strategy
 - 3 quality attributes they will prioritize
 - A risk mitigation approach
 - A basic timeline structure
-

Learning Outcomes

By the end, students should:

- Connect theory to real-world failure
 - Understand why documentation matters
 - See how poor requirements destroy projects
 - Realize testing at the end is a fantasy
 - Appreciate risk management
-

Solution

Failure Analysis & Root Causes

Identified Failures and Root Causes

Problem	Root Cause
System crashes during peak hours	No performance testing, poor capacity planning
Doctors refuse to use system	Poor usability, no stakeholder involvement
Patient data exposed	Weak security design, no security testing
Project exceeded budget	Poor estimation, scope creep, no risk management
Timeline exceeded	No formal planning, uncontrolled requirement changes
Frequent feature changes	No change management process
Testing rushed at end	Waterfall-like behavior without discipline

Deep Insight

The real failure was not “technical.”

It was **process failure**.

- No proper Requirements Engineering
- No risk assessment
- No validation with end users
- No iterative feedback
- No documentation baseline

This is a classic case of unmanaged scope + weak governance.

Mapping to Quality Attributes & SDLC Phases

Problem	Quality Attribute Violated	SDLC Phase Ignored
Data breach	Security	Design & Testing
System crashes	Reliability / Performance	Design & Testing
Doctors reject system	Usability	Requirements
Cost overrun	Maintainability / Efficiency	Planning
Delays	Project Management Constraint	Planning
Changing requirements	Flexibility	Requirements & Change Management

Key Explanation

- Security must be designed early (Secure by Design principle).
- Usability failures always originate in weak requirements gathering.
- Performance problems are architectural issues — not late testing problems.

Most Suitable SDLC Model

Recommended Model: Spiral Model

Why?

Healthcare systems are:

- High risk
- Security-sensitive
- Regulated
- Complex

The Spiral Model emphasizes:

- Risk identification
- Iterative refinement
- Continuous validation
- Early risk mitigation

Each iteration would:

1. Identify major risks (e.g., data breach)
2. Build a prototype
3. Validate with doctors
4. Perform risk evaluation

Alternative Acceptable Answer: Agile (with discipline)

Agile would work IF:

- Strong product owner involvement
- Regular stakeholder demos
- Clear sprint reviews
- Security integrated into each sprint

Agile without documentation becomes chaos. That likely happened in the scenario.

Rescue Plan (Consultant Strategy)

Step 1: Immediate Stabilization

- Freeze new feature additions
- Perform security audit
- Conduct performance testing
- Patch critical vulnerabilities

Step 2: Re-Requirement Phase

- Conduct structured stakeholder interviews
- Create formal SRS document
- Prioritize features

Step 3: Adopt Risk-Driven Development

- Implement Spiral or Hybrid Agile-Spiral approach
- Introduce change control board
- Define acceptance criteria for each iteration

Step 4: Quality Attribute Prioritization

Top 3 attributes for healthcare:

1. Security
2. Reliability
3. Usability

Step 5: Testing Strategy

- Unit Testing
- Integration Testing
- Security Testing
- Load Testing
- User Acceptance Testing

Testing is not a phase. It is a continuous activity.